Final exam preparation for Group 15

Group 15 • Final assessment preparation • 3 November 2026

The strongest defense of this project is that its design decisions are tied to correctness invariants and measured results, including a failed stress run that led to a concrete database optimization. The remaining route to the rubric's Excellent band is to restore a working demo, demonstrate variable event and HTTP workloads, and make every member able to explain the complete system.

Current status on 3 November 2026: all Docker containers have been removed, so the application and panels are stopped. The named data volumes remain, but their contents have not been recovered or integrity-checked. The source matches the October implementation snapshot. Today's unit run passed 42 tests; the ten integration tests were deselected. Current audit.

The official rubric, page 1, allocates 70% to the final demo and 30% to architecture. It requires a working demo for grading and permits questions to be redirected to any member. A report cannot replace the live performance requirements.

Read in this order

Document	Use
Rubric and readiness	Understand exactly what Excellent requires and which gaps matter first
Architecture and design defense	Explain responsibilities, alternatives, correctness, scaling, and bottlenecks
Performance and development history	Learn the measured results, failure-to-fix story, and limits of the evidence
Oral exam handbook	Practise short answers and technical follow-ups; includes worked examples
Team rehearsal and presentation	Prepare all three members, rotate questions, and rehearse a coherent presentation
Recovery and demo runbook	Recover safely after the disk cleanup and verify the correct demo mode
Exam quick reference	Review the essential facts immediately before the exam

Editable Markdown, printable PDF versions, and a combined study book are included. The architecture/performance reports are suitable source material for the submission; the rehearsal and readiness documents are internal preparation. Use the official submission format and length if separately specified.

If you only have one hour

Spend ten minutes together on the quick reference and rubric, twenty minutes tracing one measurement and two failure cases on a whiteboard, twenty minutes answering questions at random, and ten minutes agreeing the demo recovery and evidence responsibilities. Each person must explain an unfamiliar component without handing the question to a specialist.

The Computing Science member can lead the concurrency/correctness rehearsal, the Data Science member the experiment-design/performance rehearsal, and the Business Informatics member the requirements/cost/operations rehearsal. These are suggested preparation roles, not a claim about who implemented earlier work. All three must understand all three areas.

Evidence and scope

The project already includes an exhaustive implementation report, a complete code patch, and its source manifest. These remain the code-level appendix. This study pack explains how to defend that work against the rubric without repeating 10,000 lines of source.

Historical results come from the dated benchmark files. The final instructor-tool run delivered 610 of 610 required notifications, maximum 5.641 seconds, 97.310 achieved HTTP requests/second, and 202 correct history comparisons at 100 experiments × 10 sensors × 1 Hz. This establishes that workload, not an unlimited or currently running service.

The full source/evidence index is in the performance report. The proposed extra experiments, operational improvements, and role assignments are clearly identified as future work. Do not present them as completed implementation or observed results.

Regenerating the printable reports

The build script reads Markdown and archived benchmark JSON to create the diagrams, charts, individual PDFs, and combined book. It does not operate the application.

Install reportlab, pymupdf, and markdown-it-py in a separate documentation environment, then run:

```
python reports/final-exam/build_reports.py
```

The report build and document checks are recorded under evidence/. Application dependencies and source were not changed to generate the pack.

Rubric and readiness assessment

Group 15 • Final assessment preparation • 3 November 2026

The rubric rewards a working system under load and an explanation of why it behaves that way. Group 15 has strong evidence for data correctness, durable processing, and a successful local workload. Excellent is not yet established for the professor's variable workload, and restoring the stopped deployment is the first practical priority.

The official assessment

The Group Assignment Rubric INFOMCEC 2026–2027, page 1, distinguishes the following bands. These are paraphrases of the rubric, not additional grading rules.

Component	Fair	Good	Excellent
Final demo, 70%	Working implementation and consistent data meeting the interfaces and requirements	Also passes variable workload stress with notification latency below 10 seconds	Also keeps up with the variable HTTP workload
Architecture assessment, 30%	All interfaces implemented, data consistency, and separation of concerns	Distinct responsibilities and separation based on scalability	Also pinpoints different bottlenecks when load is applied

A working demo is a prerequisite for the assignment to be graded. Individual grades can differ because of contribution and knowledge. Questions may be redirected to any group member; specialization does not excuse an inability to explain another service.

The rubric's resit note says an additional resit beyond the midterm/final opportunities, if granted, is restricted to a passing grade for both demo and architecture. If that special situation applies to your group, preparation cannot remove the policy ceiling. The ordinary final assessment is the target assumed here.

The rubric does not give a numeric score conversion for each band. Do not calculate a predicted final grade from these descriptors.

Criterion by criterion

Excellent-level requirement	Available evidence	Current position and next action
Demo works	October local readiness, instructor validation, and connection artifacts	**Blocked operationally now:** zero containers after cleanup. Restore and verify before relying on the demo
Consistent data	202 complete endpoint comparisons in the final instructor run; transaction/replay/cache tests	Strong historical evidence. Repeat integration and instructor correctness checks after recovery
Below 10 seconds under variable load	All 610 notifications in the final local instructor run, maximum 5.641 seconds	Proven for one stated load profile. Add changing rates/bursts and strict maximum checks
Keeps up with variable HTTP load	97.310 achieved against 100 offered HTTP/s in that run	Strong baseline, incomplete variable-rate evidence; earlier 200-HTTP/s profile missed the local throughput criterion
All external interfaces	Kafka Avro input, required history API, notification API	Implemented. Recheck current course connectivity and external API reachability after recovery
Distinct responsibilities	Scheduler, stateless executors, Data, gateway, notifier	Explain ownership and independent scaling, plus the extra serialization/network cost
Pinpoints different bottlenecks	State-write round trips led to a specific fix; replica matrix exposed a shared-resource ceiling	Present these observations and show how metrics distinguish CPU, database, partition, cache, and notification constraints
Individual understanding	Source, test artifacts, worked cases, question bank	Each member must demonstrate it in rehearsal; reports cannot establish individual competence

Sources: final stress, failed stress, sizing matrix, current audit.

What the current results support

A defensible statement is:

> *We implemented all required interfaces and preserved consistent histories in our tests. On a two-vCPU VM, our final local run of the instructor's original tools handled 100 experiments with 10 sensors at 1 Hz, delivered all 610 required notifications within 5.641 seconds, and achieved 97.310 successful HTTP requests per second against 100 offered. We can explain both the database optimization that improved this run and the shared-resource limit observed in other profiles.*

The final run used one completed warmup before the 100 concurrent experiments. Its 202 history comparisons equal two endpoints times 101 total completed experiments. The average number of notifications varies with the simulator's random temperature sequence.

It would be inaccurate to say that all professor workloads have passed, that 200 HTTP/s is sustainable on this host, that the service is currently live, or that external notification delivery is exactly once.

Priority actions

Priority 0 Restore the demonstration

Recover the core deployment with the same Compose project and named volumes. Verify PostgreSQL readiness and old experiment data before making stronger preservation claims. Keep enough disk headroom for rebuilding and retained data; the cleanup left approximately 3.5–3.9 GiB available depending on observation time.

Check API health, Kafka partition assignment, outbox status, and the selected topic/destination. For the professor's demo these are the shared experiment topic and the external notifications endpoint. A local

grading-panel pass uses a different topic and destination.

Done means: a fresh readiness record, successful required endpoint queries, correct configuration, and current course preflight. Follow the recovery runbook. Container creation alone does not complete this action.

Priority 1 Close the workload evidence gap

Repeat the original passing profile after recovery, then vary event and HTTP rates over time and retain raw results. Test a rising step, a burst, a sustained plateau, and a return to baseline. Examine whether queue depth returns to its former level rather than growing across repeated steps.

Retest the older failing 200-HTTP/s and twenty-sensor combined profiles on the final implementation. Those October 7 runs predate the October 9 batch-write optimization. They establish earlier limits, not a measured post-optimization ceiling.

Use strict notification maximum **less than 10 seconds** and zero missing/invalid notifications. The existing custom harness checks maximum less than or equal to 10, and the course checker counts delays greater than 10. Exactly 10 is therefore accepted by those local gates but not by the rubric's wording. The successful recorded runs are below 10, so this does not invalidate their actual values. Tighten a future automated gate or explicitly check maximum less than 10 when evaluating a new run.

Our 95% achieved-HTTP threshold is an internal acceptance rule; the professor has not specified that exact percentage. Show offered and completed rates, validation errors, latency, queue trend, and recovery time rather than treating the local boolean as the whole grading decision.

Done means: changing-load traces with correct data, strict latency results, and evidence that requests and queues keep up on the selected deployment.

Priority 2 Make bottleneck explanations observable

For each tested load step, align event rate, HTTP rate, queue length, CPU, executor occupancy, Data latency, outbox age, and validation outcome on one time window. State a hypothesis, identify a metric that would support it, make one targeted change, and compare.

The existing metrics show useful symptoms. Detailed SQL statement timing, pool-wait timing, complete Kafka lag, and per-partition hot-spot monitoring are not all implemented as dedicated metrics. Use database/consumer inspection when needed, or add those instruments as future work. Do not describe planned instruments as existing dashboards.

Done means: all three students can distinguish at least two bottlenecks and explain why the same scaling action does not fix both.

Priority 3 Prevent another storage outage

The disk incident stopped every service, even though named volumes survived. Separate regenerable images/build cache from valuable database and monitoring volumes. Preserve data before cleanup, define storage headroom, measure growth, and alert on filesystem usage. A bounded Redis cache does not bound PostgreSQL history, outbox, Kafka, Docker cache, or monitoring storage.

The current application has no history archival policy or filesystem-full alert rule. Propose a policy compatible with the requirement to retain an experiment's history; deleting older measurements merely to recover space would change product behavior.

Done means: an agreed preservation/recovery process, a monitored capacity threshold, and sufficient space for the demo and its test evidence.

Priority 4 Prove individual understanding

Use the random redirection drills in the team plan. Each member should draw the architecture, calculate a mean/notification example, explain a crash window, read one performance result, identify a bottleneck, and state a limitation.

Record actual contributions with file/commit/test evidence. Assigning someone a presentation topic now does not prove they implemented it earlier. Discuss genuine study, validation, analysis, documentation, operations, and code contributions accurately.

Improvements ordered by value and risk

Improvement	Why it helps the rubric	Evidence required before claiming success
Restore current deployment	Mandatory working-demo gate	Fresh health, connectivity, correctness, and live-mode checks
Strict deadline gate	Matches the actual wording	Boundary case at exactly 10 rejected; measured maximum still below 10
Repeat and vary offered rates	Directly targets Excellent demo	Multiple traces including rise, plateau, burst, and drain
Diagnose database versus CPU pressure	Directly targets Excellent architecture	Correlated metrics and a controlled intervention
Disk monitoring and preservation plan	Prevents another demo outage	Capacity trend, alert check, recovery rehearsal
Calibrate cost inputs	Strengthens the resource/cost argument	Actual billed rates and an explicit scope for the estimate
More executors	Helps only when executor work is limiting	Improved combined latency/throughput without downstream overload
Multiple Data processes	Potential future capacity, with correctness risk	Shared cache invalidation or removal of local metadata cache, plus concurrency tests
Larger or separated infrastructure	Potential resource relief, with cost tradeoff	Confirm allowed deployment constraints; compare performance and cost

The goal is a convincing chain from requirement to design to measurement to revision. Additional components or pages do not compensate for a stopped demo or unsupported performance claims.

Architecture and design defense

Group 15 • Final assessment preparation • 3 November 2026

The application separates event intake, computation, durable storage, public reads, and notification delivery because they have different ordering, failure, and scaling requirements. Its central correctness decision is to commit experiment state, history, notification obligations, and Kafka progress together in PostgreSQL before acknowledging Kafka.

This architecture passed a specific local workload, but its scaling is constrained by a shared two-vCPU host, a single Data process, PostgreSQL, and Kafka partition ordering. Understanding those boundaries is part of the design defense.

Requirements that shaped the design

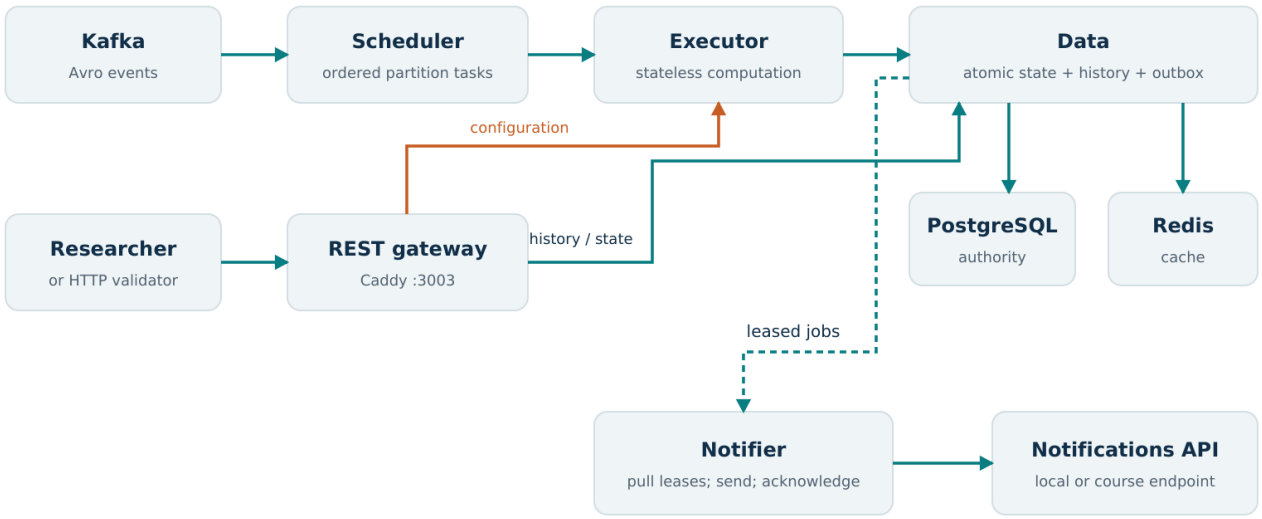
The laboratory controls temperature elsewhere. This service only observes it. A measurement is the mean of all allocated sensors for one experiment and measurement ID. It cannot substitute the most recent readings from different sampling instants.

Configuration makes the experiment queryable immediately. Stabilization can produce a Stabilized notification. Carry-out begins only on the researcher's start event and contributes the history exposed by the required endpoints. During carry-out, only the first outside measurement of a continuous excursion triggers OutOfRange. Termination stops accepting new measurements but retains past history.

The two required read endpoints return ordered arrays of timestamp/temperature pairs. Time bounds are inclusive. The outside-history endpoint returns all outside measurements, although only the start of each excursion produces a notification. Those are separate products, so they must not share the same filtering rule. Assignment requirements.

Components and ownership

Application responsibilities and data flow



PostgreSQL owns durable state. Redis is disposable. The notifier pulls jobs from Data.

Component	Owns	Does not own	Scaling boundary
Scheduler and decoder	Kafka membership, per-partition queues, ordered batches, executor selection	Durable experiment state	Sequential work within a partition; consumer membership and host resources

Component	Owns	Does not own	Scaling boundary
Stateless executor	Validation and state-machine computation for a task	Long-lived authoritative experiment data	Capacity eight by default; more replicas share downstream Data
Data	PostgreSQL transactions, history queries, Redis responses, outbox leases	Sending external notifications	One process owns local metadata-cache invalidation; database is shared
User REST gateway	Public routes, transport forwarding, docs, gateway metrics	Domain calculations or database ownership	Forwarding capacity and downstream response time
Notification dispatcher	Claiming jobs, HTTP delivery/retry, outcome acknowledgement	Deciding which measurement needs notification	Delivery concurrency, external endpoint, durable outbox
PostgreSQL	Authoritative states, aggregates, progress, jobs, rejected input	Disposable rendered query cache	CPU, I/O, locks, connection capacity, storage
Redis	Disposable versioned history responses	Authoritative history or experiment state	Memory bound and cache hit ratio

The application workers are Python. Kafka, PostgreSQL, Redis, and Caddy are existing infrastructure components. Python's suitability is supported by measured end-to-end results and asynchronous I/O, not by a claim that Python is universally the fastest runtime.

A measurement from Kafka to notification

For experiment E, configuration lists sensors A, B, and C. Messages for a measurement M share an ID, timestamp, and opaque hash. The decoder reads the Avro object container and its record_name Kafka header. The scheduler preserves their partition order and sends a bounded task batch to a reserved executor.

The executor loads a consistent snapshot containing E's state/version, the partition checkpoint, and relevant completed measurement identities. It adds A, B, and C to M's pending group. Only when all configured sensors have arrived does the pure engine compute the mean, classify the range, and create a history point and, when appropriate, an outbox job.

Data checks the proposed state versions and checkpoint inside one transaction. A successful commit records all effects together. The scheduler can then acknowledge the offset after the last processed event. The notifier later leases the job, sends the required payload with the original hash as cipher_data, and acknowledges its result.

The source producer uses an Avro started-event schema inside termination messages. The record_name header correctly identifies termination, so the decoder treats the header as authoritative. This is compatibility with the supplied implementation, not a modification of its source. Decoder, engine.

Design decisions and alternatives

Separation into services

A modular monolith would reduce network calls, serialization, operational overhead, and the number of failure points on a small VM. It is a credible alternative.

We chose explicit service boundaries to match the brief and to isolate responsibilities with different load patterns: Kafka consumption must preserve order, computation can be replicated, history reads benefit from caching, and slow notification delivery should not block ingestion. Internal HTTP makes these boundaries inspectable and independently deployable.

The cost is additional hops and shared-host contention. The replica experiments demonstrate this cost: extra executors did not improve combined HTTP capacity in the measured configuration. We therefore selected one executor instead of claiming that more containers automatically meant more scalability.

Kafka for intake and replay

Kafka is the required interface and offers partition ordering, retained input, and independent consumer progress. We use these properties to recover from process loss. Kafka alone does not make PostgreSQL writes or an external HTTP call exactly once.

The scheduler uses one active task per partition. This preserves order without permanently binding an experiment to one executor. The producer must keep each experiment on a consistent partition. Scaling the consumer group can divide partitions, but cannot parallelize a single partition without another ordering mechanism.

Adding partitions to a live key-based topic is also a design change: reassignment of a key can break the assumed ordering across old/new partitions. It should not be presented as a risk-free scaling switch.

Stateless executors

An executor's local counters and HTTP connections are temporary. Authoritative partial sensor groups, prior range state, lifecycle status, and versions live in PostgreSQL.

This makes worker replacement and load-based dispatch practical. The extra database round trips are the tradeoff. The later state-write optimization reduces that overhead without moving correctness-critical state back into workers.

PostgreSQL as the authority

A relational database provides atomic multi-table transactions, uniqueness constraints, indexed history, row locks, and optimistic version checks. One transaction can link a measurement to its notification obligation and durable progress.

Redis-only state would need another durability and transaction strategy. A document store or stream-state framework could be considered, but would still need to satisfy replay, query, and external-delivery semantics. PostgreSQL fit the measured workload and kept these guarantees in a place the team could test directly.

The five tables are experiments, checkpoints, measurements, notification_outbox, and rejected_events. JSONB stores evolving experiment state; relational columns and indexes support common history queries and uniqueness. This is a hybrid representation, not storing every query field in an unindexed document. Schema.

Redis as a disposable cache

Repeated HTTP history queries can ask for identical content at one experiment version. Caching a serialized response avoids another database query and repeated response construction.

Cache keys include the experiment version and query mode/bounds. Metadata is invalidated around every write, and an epoch plus active-writer count prevents a concurrent cache fill from preserving an obsolete version. A cache miss retrieves metadata and points in one SQL statement so they come from one database snapshot.

Redis failures fall back to PostgreSQL after a short timeout and temporary bypass. A Redis outage therefore changes cost/latency rather than deleting history. The integration tests use a real database, an intentionally unavailable Redis endpoint for fallback, and a small cache double for deterministic invalidation behavior; the separate recovery exercise stops the actual Redis container. Data tests, recovery.

The in-process metadata cache means multiple independent Data processes are currently unsafe as a casual scaling change. They could disagree about the newest version. Remove that local cache or coordinate invalidation before horizontally scaling Data.

Durable outbox for notifications

Sending an alert before committing its measurement risks an alert about state that later rolls back. Committing a measurement and only then adding an in-memory notification risks losing the alert on a crash.

The transaction writes both measurement and outbox entry. A separate dispatcher can retry delivery after process loss. Database uniqueness prevents duplicate jobs for the same experiment/measurement/type.

This guarantees durable notification obligations, not exactly-once external acceptance. A receiver can accept a request just before the sender crashes or loses its acknowledgement. Redelivery is then possible. The Idempotency-Key helps only if the receiver honors it. Notifier.

Least-busy scheduling

Each worker reports active tasks and capacity. The scheduler also makes an immediate local reservation so several dispatches do not all select a worker based on the same stale load observation. Effective load is the maximum of reported active work and local reservations.

Least-busy dispatch is simple and fits roughly comparable tasks. Batches can still differ in event count, sensors, database work, and cache effects; the scheduler is not an exact cost predictor. Worker-side capacity enforcement and retries are the final protection against stale observations. Discovery.

Batching and backpressure

Batching amortizes network and database overhead. Larger batches can improve throughput while increasing task latency, lock duration, fairness issues, and replay cost. The default batch size is 256 events; the accepted task maximum is 1,024.

The scheduler pauses intake at 8,192 queued/in-flight events and resumes below half that level. Up to one fetched batch of 512 can pass the pause boundary. Pausing bounds application buffering but shifts waiting into Kafka. It does not guarantee that a ten-second deadline will still hold.

At a hypothetical processing rate of 1,000 events/second, 8,192 queued events represent about 8.2 seconds of processing ahead of a new event, before other delays. That conditional calculation shows why queue age matters; it is not a measured service-rate guarantee.

Public gateway and transport choices

Caddy forwards public reads directly to Data and configurations to an executor. This reduces Python proxy overhead and keeps internal commit/claim routes off the public port. The Python API module defines OpenAPI and remains a development entry point.

Internal HTTP uses a pooled aiohttp transport with HTTPX response/error types. Direct ASGI metrics avoid extra middleware tasks. PostgreSQL renders history JSON, and cache I/O does not hold a database connection. These are implemented overhead reductions, but individual speedups were not isolated for each in the preserved experiments.

Explicit, bounded autoscaling

The host-side autoscaler adds a replica after sustained backlog/occupancy and removes one after a longer idle period. It has configurable bounds and dry-run support. It is not started automatically by the normal Compose deployment.

The policy responds to executor demand, not a complete CPU/database model. If Data or the VM is already saturated, scaling executors can worsen contention. The correct scaling target comes from diagnosing the limiting stage first. Scaling policy.

Correctness invariants

Invariant	Mechanism	Failure it prevents
Mean uses one complete measurement group	Experiment/measurement key; unique allocated sensors; matching timestamp/hash	Mixing time instants, double-counting a sensor, premature alerts
Phase and measurements stay ordered	Per-partition sequencing and state-machine timestamp/phase checks	Carry-out/stabilization confusion
State, history, jobs, and progress agree	One PostgreSQL transaction	Committed progress with missing effects
Concurrent writers cannot silently overwrite	Expected state versions and locked checkpoint	Lost updates
Replayed completed groups do not add history	Measurement primary key and completed-ID lookup	Duplicate aggregates at new Kafka offsets
A worker only acknowledges its current job lease	Lease-token predicate	A stale acknowledgement overwriting a newer owner
Reads after an acknowledged write use current metadata	Local writer fences and versioned cache keys	Returning an obsolete cached history indefinitely

Invalid Kafka input is durably quarantined so later events can progress. Quarantine is visible and counted; it does not mean invalid or missing input has been repaired. A permanently missing sensor can block subsequent groups in that experiment, and a phase transition with incomplete groups is rejected.

Crash cases worth explaining aloud

Failure point	Consequence	Recovery
Before the Data transaction commits	No committed state/history/job/progress for that task	Kafka task retries
After Data commits, before executor response or Kafka acknowledgement	Durable effects exist; Kafka may replay	Checkpoint/completed identities suppress repeated effects
During an optimistic version conflict	Entire proposed transaction rolls back	Executor reloads and recomputes
Executor restarts with an incomplete group	Local process memory disappears	Persisted partial readings are reloaded
Redis is unavailable	Cache lookup fails quickly	Query authoritative PostgreSQL
Notifier crashes after claiming, before sending	Job temporarily leased	Lease expiry allows retry
Receiver accepts, sender loses acknowledgement	Possible repeated external delivery	Receiver-side idempotency is needed for unique acceptance
VM or disk becomes unavailable	All cohosted services can stop	Operational recovery; microservice boundaries do not provide host redundancy

A fifteen-second lease supports eventual recovery but can itself exceed the ten-second alert deadline after a crash. The implementation therefore does not guarantee the latency SLA through every failure even when it preserves the obligation to notify.

Bottlenecks and what to observe

Limiting stage	Supporting symptoms	Distinguishing check	Targeted response
Shared CPU	High node CPU with both HTTP latency and processing queue rising	Compare executor count at the same workload; inspect process CPU	Reduce overhead or move/resize resources if allowed
Data writes/database	Commit path slow, queue and executor occupancy high, database waits	Observe query/pool/lock timing; compare batched writes	Batch operations, tune queries/indexes, then remeasure
History reads	HTTP slow while intake remains healthy, low cache hit ratio or large responses	Compare warm/cold cache and short/long histories	Query/cache optimization; respect the required full-response API
Hot Kafka partition	Uneven progress despite free executor capacity	Inspect lag by partition	Fix key skew or redesign ordering; adding idle workers is insufficient
External notification endpoint	History current but pending jobs/age/retries rise	Compare outcome codes and external response delay	Bounded concurrency/backoff and endpoint recovery
Redis outage	Unavailable-cache counter rises and database read load increases	Check cache metrics and Data queries	Restore cache; preserve database fallback
Storage	Filesystem growth and failed writes/startup	Check actual filesystem, volumes, logs, build cache	Preserve data, recover capacity, define retention/archival
Poison or incomplete input	Rejections or pending groups rise without expected aggregates	Inspect rejection reason and sensor completeness	Correct producer/input; do not invent readings

Only the state-write improvement and shared-resource replica behavior have direct before/after workload evidence in the preserved reports. Other rows are testable diagnostic explanations, not claims that every bottleneck occurred.

Capacity and cost reasoning

Let E be concurrent experiments, S sensors per experiment, and T seconds per sample. Sensor input rate is $E \times S / T$. Aggregate production is approximately E / T, independent of S once a group completes. HTTP traffic adds separate work and can grow with response size, history length, and cache misses.

With sixteen partitions, K executors, and capacity C=8, a simple upper bound on simultaneous partition tasks is $\min(16, K \times 8)$. This is a concurrency bound, not a throughput formula. Service time, partition skew, CPU, and database contention determine achieved throughput.

At 100 experiments and 1 Hz, one day of continuous carry-out creates 8.64 million aggregate rows before considering stabilization, indexes, outbox, retained Kafka input, and monitoring. Arbitrary duration therefore requires a storage plan even though only 100 experiments run concurrently.

The implemented cost metric is configured VM dollars/hour plus measured PostgreSQL size in decimal GB times configured storage dollars/GB/month divided by 730. It omits several billed resources and defaults to unconfigured zero. It is useful for an explained estimate, not a provider bill. The October audit recorded cost_configured=0, and no .env.monitoring file was found in the current check.

What we would change next

First restore operational readiness and measure changing workload on the final code. Then improve whichever stage is demonstrably limiting. Add precise pool/query/partition timing if current metrics cannot distinguish the cause.

For stronger resilience, plan backups, storage growth alerts, retention or archival that preserves required history, and deployment across failure domains. For safe Data replication, replace process-local invalidation with a shared strategy. For externally unique notification acceptance, agree an idempotency contract with the receiver.

These are proposed extensions. The existing design's merit is its explicit and tested correctness boundaries, not that it has already become a highly available production platform.

Performance results and development history

Group 15 • Final assessment preparation • 3 November 2026

The main measured improvement was replacing individual PostgreSQL state-write round trips with pipelined, version-checked writes. Under the same instructor workload settings, maximum notification latency fell from 10.716 to 5.641 seconds and achieved HTTP throughput rose from 86.848 to 97.310 requests per second. Both runs already returned correct complete histories.

This is a before/after observation from short, stochastic runs. It supports the chosen optimization and motivates repeat testing; it is not an isolated causal estimate or proof of every future workload.

Test environment and workload definitions

The historical measurements ran on Ubuntu 24.04, two AMD EPYC 7571 vCPUs, approximately 3.756 GiB RAM, and Python 3.12. Kafka, PostgreSQL, Redis, the application, monitoring, and test generators shared the host. Measuring them together captures the actual local footprint while also making the load generator and observability services potential competitors for CPU and disk.

The deterministic harness uses the supplied Avro schemas and independently constructed expected data. The later course harness builds the original Rust producer, notification service, and HTTP validator without editing their logic. It provisions their ground-truth database and adapts the repository's original dashboard to local datasources.

These are separate forms of evidence. The Python notification double validates synthetic test truth; it is not the real cryptographic instructor validator. The October 9 runs use that real validator locally. Neither type measures the complete external network path of a professor-run demo.

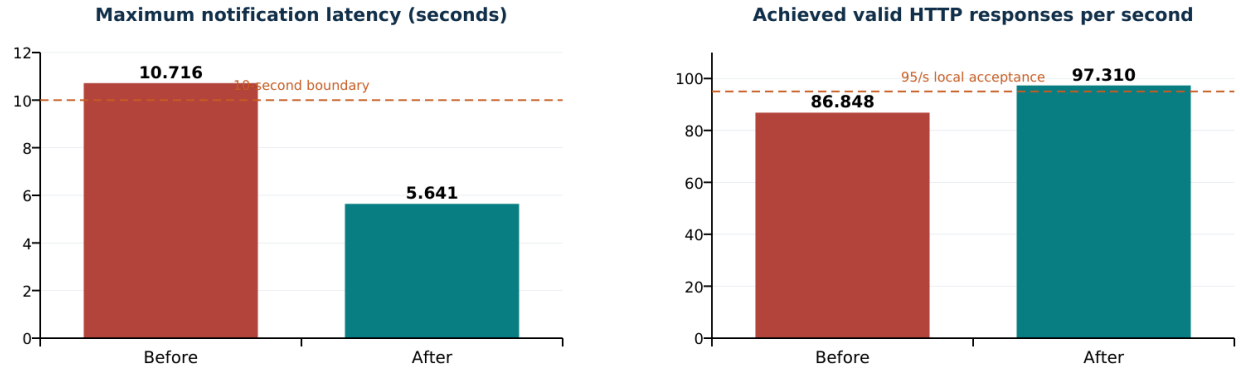
The final instructor workload

Parameter	Value
Concurrent main experiments	100
Sensors per main experiment	10
Sample period	1 second
Main carry-out samples	60 per experiment
Offered HTTP rate	100 requests/second
Required HTTP observation	At least 60 seconds
Warmup	One completed experiment before the main experiments
Executors	One, with capacity eight
Notification delivery	0.1-second empty-poll interval, concurrency 32
History comparison	Both complete endpoints, timestamp/temperature tolerance below 0.001

The sample input rate during main carry-out is approximately 1,000 sensor events/second, producing 100 aggregate points/second. The 202 complete history comparisons cover 101 experiments including the warmup. These comparisons supplement the continuous instructor HTTP traffic.

Before and after the database change

Instructor stress test before and after pipelining



100 experiments x 10 sensors x 1 Hz; 100 offered HTTP/s. Historical local runs, 9 October 2026.

Different random temperature paths; these are observed before/after values, not repeated A/B estimates.

Measure	Initial stress	Final stress
Required notifications	632	610
Received notifications	632	610
Missing notifications	0	0
Notifications above 10 seconds	3	0
Maximum notification latency	10.715822 s	5.640533 s
Achieved HTTP rate	86.847621/s	97.309762/s
HTTP validation errors	0	0
Complete history comparisons	202 correct	202 correct
Local combined result	Fail	Pass

The recorded maximum fell by **47.36%**, and achieved HTTP throughput rose by **12.05%**, calculated from these two reports. The final maximum had **4.359 seconds of margin** below ten seconds. This margin is not a promise under overload or failures.

The initial commit loop awaited fetchval once for each changed experiment. The replacement sorts experiment IDs, groups adjacent inserts/updates of the same kind, and uses fetchmany. It checks that every proposed state change returned a successful version row; otherwise the whole transaction rolls back.

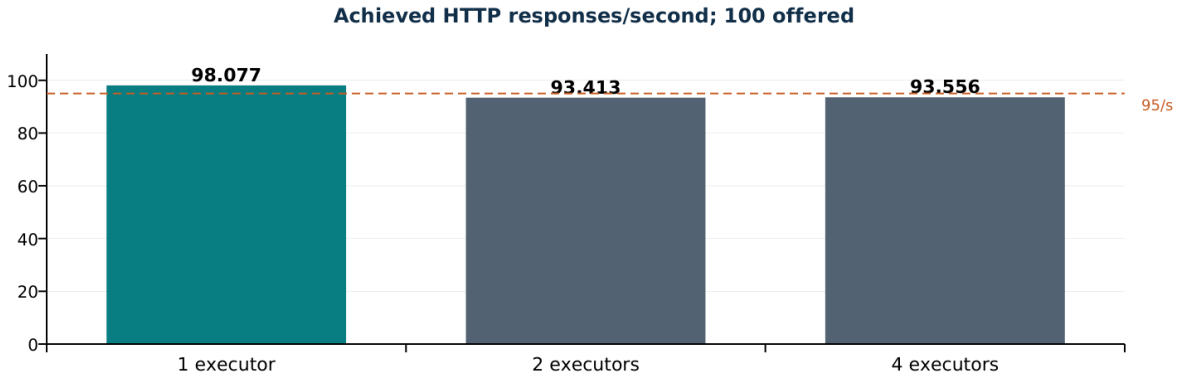
Sorting before grouping preserves global row-lock order. Checking every return preserves optimistic concurrency. The added test deliberately makes the middle of three versions stale, verifies all three writes roll back, corrects the version, and verifies all three advance. Performance improved while the consistency rule stayed intact. Exact code, regression test.

The notification count differs because the original simulator generates random temperature paths. Comparing workload parameters is appropriate; treating 632 and 610 as identical event sequences is not. The record does not include a profiler time breakdown or repeated randomized A/B runs, so the exact size of the optimization's isolated effect remains unmeasured.

Earlier executor and polling experiments

The October 7 matrix used 100 experiments, 10 sensors, 1 Hz, 30 carry-out samples, and 100 offered HTTP/s. Every row produced all 875 expected notifications and correct histories.

More executors did not improve combined capacity on this VM



100 experiments x 10 sensors x 1 Hz, 0.1-second notification poll. Historical local runs, 7 October 2026.

The dashed line is our 95% acceptance criterion. Notification deadlines and data correctness passed in all three.

Executors	Poll seconds	Achieved HTTP/s	HTTP p95 seconds	Notification p99 seconds	Notification maximum seconds	Result
1	0.1	98.077	0.689	3.587	4.025	Pass
2	0.1	93.413	3.340	3.222	3.637	HTTP rate below local criterion
4	0.1	93.556	3.229	3.098	3.311	HTTP rate below local criterion
1	0.5	98.070	1.798	5.959	6.768	Pass
1	1.0	97.587	1.169	3.322	3.991	Pass

One executor was the smallest passing count. The 0.1-second poll had the lowest observed HTTP p95 among the passing rows, so it became the default. The short runs do not establish that this interval is universally optimal.

Extra executors improved some notification/queue behavior but worsened combined HTTP performance on the shared host. This is evidence against using replica count as a substitute for bottleneck diagnosis. CPU and shared Data/database contention are plausible mechanisms; per-stage profiling would distinguish their individual contributions.

The HTTP p95 values above belong to the deterministic October 7 tests. Do not attach 0.689 seconds to the later October 9 instructor run, whose JSON does not report that same p95 statistic.

Earlier capacity boundaries

Sensors	Offered HTTP/s	Achieved HTTP/s	HTTP p95 seconds	Notification maximum seconds	Result
20	0	Not offered	Not measured	3.168	Pass
10	200	169.703	9.387	5.186	HTTP rate below local criterion
20	100	90.687	6.143	7.293	HTTP rate below local criterion

These profiles used one executor and a 0.1-second notification poll. Histories and notification sets were correct, and notification delays stayed below ten seconds. The combined result failed where completed HTTP throughput did not keep up with the offered rate according to our 95% rule.

The twenty-sensor/100-HTTP profile reached a queue peak of 7,440 events. This warns that notification headroom can disappear before data is lost. The 200-HTTP run completed requests but needed drain time and had high tail latency. “No HTTP errors” is therefore weaker than “keeps up.”

These runs preceded the batch-write change. They should be repeated on the final code before claiming its upper capacity.

How correctness and recovery were established

The historical full suite passed 52 cases after the batch-write regression: 42 unit and 10 integration. It covers Avro compatibility, sensor averaging, phases, boundaries, duplicates, replay, offset ordering, version rollback, notification leases/retries, query contracts, and metadata-cache races.

The integration suite uses a real PostgreSQL database, the Data application's HTTP handlers, and the deployed gateway where relevant. Redis fallback is deliberately exercised with an unavailable address; a small cache double makes selected cache behaviors deterministic. The separate end-to-end recovery exercise verifies actual Redis loss.

That recovery exercise preserved four history points and three expected notifications through incomplete sensor persistence, executor restart, new-offset logical replay, two injected notification 503s, scheduler restart, Redis outage, and Data/PostgreSQL restart. It proves these specific cases, not arbitrary host-loss availability or every crash window.

On 3 November the unchanged source passed all 42 unit cases again in 1.88 seconds. The ten integration cases were deselected because containers were absent. Fresh output.

Development decisions and obstacles

Problem or finding	Change or response	Evidence and lesson
Sensor readings arrive separately	Persist partial groups and wait for all allocated sensors	A mean must represent a complete sampling instant
Producer termination schema differs from header	Decode by record_name and test the quirk	Inspect actual upstream behavior while preserving its validator
Replays and worker loss can duplicate work	Atomic checkpoint/state/history/outbox plus identity deduplication	Kafka progress and application effects must agree
Concurrent cache fills can retain old metadata	Version keys and writer/epoch fences	Caching is part of correctness, not only speed
More executor replicas did not improve combined throughput	Select one measured passing executor; retain a bounded scaling mechanism	Shared-resource limits can dominate parallelism
External Kafka failed despite reachable ports	Demo-specific hostname-matching exception; keep CA/client authentication and add a preflight probe	Separate network reachability, server identity, authentication, and assignment
Published notification image lacked current public-key configuration	Build the original Rust tools and generate local test credentials	Independent validation was available from source
Instructor testing needed an explicit result	Add nonzero-load gates, complete-history checks, result JSON, and one extra dashboard panel	Green/no-data alone is not a meaningful test
First full instructor stress missed the deadline	Pipeline state writes and add all-or-nothing version regression	A specific database operation was the optimization target
Disk exhaustion later led to killing/removing all containers	Current recovery planning and storage lessons	Process resilience does not replace disk-capacity management

The first implementation grouped several optimizations into one commit. Apart from the explicit later state-write change and its before/after runs, the record does not establish a separate date or isolated speedup for every earlier optimization.

Implemented optimizations and their tradeoffs

Implemented change	Intended saving	Tradeoff or guard
TaskLookup sends identities rather than full sensor payload	Less internal serialization and transfer	Still requires a consistent database snapshot
Batch state writes with fetchmany	Fewer sequential database round trips	Must check all versions and preserve lock order
Indexed relational history queries	Less scanning for common queries	Index write/storage overhead
PostgreSQL-generated ordered JSON	Less Python response-object work	Database performs serialization
Versioned Redis response cache	Fewer repeated history queries	Correct invalidation and bounded memory required
Single-process metadata cache	Fewer version lookups	Constrains Data replication
Caddy public gateway	Less Python proxy work	Additional configuration must match OpenAPI
Pooled aiohttp transport and direct ASGI metrics	Lower per-request overhead	Adapter correctness, including gzip, needs tests
Skip unused pool session-reset SQL	Fewer cleanup statements	Only valid while no session state is introduced; rollback is tested
Bounded concurrent notifications and immediate backlog draining	Less waiting between batches	Receiver capacity and rate limits still matter

These mechanisms are in the source. Only the batch-write improvement has the specific instructor before/after pair above; the rest should be explained mechanistically without invented performance percentages.

Stronger experiments to target Excellent

The following is a proposed test program, not completed evidence.

1. Reproduce the final baseline at least three times after recovery, recording all runs. Use the same configured rates and durations, record source/image identity, and distinguish warmup from observation.
2. Retest the earlier failed fixed profiles to find the final implementation's capacity boundary.
3. Exercise a changing HTTP sequence such as 50 -> 100 -> 150 -> 200 -> 100 requests/second while sensor traffic remains fixed. Use documented plateaus, repeat them, and capture backlog recovery.
4. Exercise event concurrency 10 -> 50 -> 100 experiments at 1 Hz, both evenly staggered and aligned at sample boundaries. Keep the requested main concurrency within 100.
5. Hold the baseline while varying sensors, history duration, cache warmth, and key/partition skew one factor at a time.
6. Run a longer soak and sample filesystem/database growth. Do not multiply a sixty-second latency result into an untested indefinite-duration guarantee.
7. Run fault recovery separately from baseline performance, then characterize which failures preserve correctness and which also preserve the deadline.

The original Rust HTTP tool supports varying batch sizes over generations. Our course runner currently pins min to the requested rate and max to rate+1, so its recorded runs are effectively fixed-rate tests. A controlled changing-rate program and per-phase acceptance calculation require explicit

harness/configuration work. Repeating several fixed-rate runs is useful capacity evidence but is not the same as one continuous variable workload.

For each phase, record offered rate, completed valid rate, errors, latency distribution and maximum, required/received notifications, maximum notification latency, queue/lag trend, outbox age, CPU/memory/disk, and recovery time. Compare completed work with the integral of the offered schedule over the same interval. Keep load-generator overhead and request retries visible.

Prefer absolute SLA results and distributions over a single average. p99 under ten seconds can coexist with a late worst-case alert. A green max-delay plot also says nothing about a notification that never arrived, so required/received counts are necessary.

Evidence index

Source	What it supports
Rubric PDF	Assessment weights, bands, working-demo gate, individual questioning
October environment	VM, dependencies, cohosted services, source hashes
Sizing matrix	Executor/poll comparisons
Twenty sensors without HTTP	Event-only boundary
Two hundred HTTP requests per second	Earlier HTTP-capacity failure
Twenty sensors and HTTP	Earlier combined-load failure
Recovery result	Specific injected-failure checks
Kafka experiment probe	Historical external assignment
Strict-hostname control	Historical strict-mode failure
Instructor smoke	Small original-validator pass
Initial instructor stress	Pre-fix late notifications/low HTTP rate
Final instructor stress	Post-fix passing stated workload
Later check of the same run	Longer HTTP observation, not another independent experiment
Instructor dashboard check	Datasource/query operation and historical panel results
Instructor source/build identity	Unmodified instructor source hashes and build context
Full historical verification	52-test result
Current readiness	Stopped deployment, surviving volume names, source comparison, unit result

The original implementation report and patch remain the full change archive. The strongest oral answers connect one of these observations to the exact mechanism and its limitations.

Oral exam handbook

Group 15 • Final assessment preparation • 3 November 2026

Every member should be able to explain the full event path, the reason for the architecture, the measured performance, and the principal limitations. Use the short answers below as reasoning models, not scripts to recite. Follow an answer with the mechanism, an observation or test, and its scope.

The rubric allows questions to be redirected. A useful answer is technically correct and specific enough that another member can continue it without changing the claim.

The shared project explanation

> *We built a temperature observability service. It consumes experiment lifecycle and sensor events from Kafka, waits for all allocated sensors in each measurement, stores the average, exposes carry-out history through REST, and sends stabilization or out-of-range notifications. A scheduler dispatches ordered partition batches to stateless executors. PostgreSQL atomically stores state, history, notification jobs, and progress; Redis caches reads. In our final local instructor-tool run, 100 experiments with 10 sensors at 1 Hz produced 610 required notifications, all received within 5.641 seconds, with correct histories and 97.310 successful HTTP requests per second. We also identified limits under heavier combined load.*

Say “historical local run” when discussing that result. After the November cleanup, the service is stopped until recovered.

Requirements and business meaning

1 What problem are we solving

We preserve the experiment's temperature history and notify the researcher at the required transitions. We do not control heaters or cooling equipment. The application supports multiple experiments with different sensors, ranges, phases, and durations.

Follow-up: The business risk is an experiment continuing outside its required conditions without timely awareness. History supports later inspection; notification supports a timely response.

2 How do we define the temperature of the room

For one measurement ID, we take the arithmetic mean of every sensor allocated to that experiment. We wait for the complete group and require readings to agree on timestamp and hash.

Follow-up: An individual sensor can be outside the range while the room mean is inside. Replacing the mean with a maximum, median, or anomaly model would change the required behavior.

3 Why not combine the latest reading of each sensor

Those readings may represent different sampling instants. The measurement ID is the grouping identity; timestamp and hash agreement provide additional consistency checks. A correct average must refer to one complete measurement.

Show: PartialMeasurement and ExperimentStateManager.apply in models and engine.

4 What happens before the experiment starts

Configuration is persisted and can be queried immediately. During stabilization we aggregate measurements and send Stabilized on entry into range, including re-entry as the course producer does. Only the researcher's explicit start event begins carry-out.

Follow-up: Stabilization aggregates are stored internally for deduplication but excluded from the two required history endpoints.

5 Do we notify for every outside sample

No. We send OutOfRange at the first outside sample of each continuous excursion. A subsequent in-range sample enables the next excursion notification. Outside history still returns every outside carry-out sample.

Follow-up: The distinction prevents repeated alerts while preserving complete evidence of how long the experiment was outside.

6 Are the thresholds and query bounds inclusive

Yes. Equality with a temperature threshold is in range. A history sample exactly at the start or end timestamp is included. A query with equal start and end can return the sample at that instant.

Show: Strict less-than/greater-than comparisons in the engine and greater-than-or-equal/less-than-or-equal predicates in Data.

7 What do the extra REST endpoints add

POST /experiments supports explicit configuration. GET /experiments/{id} shows configuration and state. The interval endpoint groups continuous excursions and reports their start/end, with null for an ongoing excursion.

Follow-up: These additions do not change the required arrays returned by /temperature and /temperature/out-of-range.

8 Why is this not a machine-learning project

The required calculation and notification rules are deterministic and fully specified. An ML model would introduce uncertainty and validation costs without solving the stated problem. Data-science work here is experimental design, workload characterization, statistical interpretation, and trustworthy measurement.

Architecture and technology decisions

9 Why these service boundaries

Consumption/order, computation, durable data, public reads, and external delivery have different failure and load behavior. Separating them prevents a slow notification endpoint from directly blocking event computation and allows independent execution capacity.

Follow-up: Boundaries add network/serialization overhead. On one VM they do not provide host-level fault isolation, and a modular monolith would be a reasonable lower-overhead alternative.

10 Why did we choose Python

The workload is largely asynchronous I/O, with Kafka/Avro/database work supported by existing libraries. The implementation is understandable and testable, and the chosen local workload passed. We support the choice with end-to-end measurement rather than language-speed rankings.

Follow-up: If profiling shows CPU-bound state computation dominates, a faster implementation could help. It will not fix a database bottleneck by itself.

11 What exactly is stateless about an executor

It does not own authoritative experiment state across tasks. A scheduled task is a bounded batch of events; an experiment spans many tasks. The executor loads a snapshot and proposes a transaction. Its local connections and active-task counter are ephemeral; incomplete groups and range state are persisted in Data.

Follow-up: Stateless does not mean no memory or no concurrency state. It means another worker can continue from durable state.

12 Why PostgreSQL instead of only Redis

We need atomic updates across state, history, notification obligations, and checkpoints, plus durable history and uniqueness constraints. PostgreSQL supplies those directly. Redis accelerates repeat reads but is not the authority.

Follow-up: Losing Redis should reduce speed. Losing the database or its volume is a materially different failure.

13 Why retain both JSONB state and relational columns

JSONB conveniently stores configuration, phase state, and variable-sized incomplete groups. Relational measurement columns support indexed timestamp/outside queries and primary-key deduplication. The representation follows the access patterns.

Follow-up: Large partial state and frequent full JSON updates have a cost; missing sensors and unbounded growth cannot be ignored.

14 Why use Caddy if there is an api.py

api.py defines the public contract and OpenAPI and supports a Python entry point. The deployed gateway is Caddy, which forwards directly to Data or an executor and serves the generated documentation with less Python proxy work.

Show: Docker stages and Caddy routes. Internal commit/claim paths are not public routes.

15 Why least-busy dispatch

Task costs are broadly comparable, so active work plus immediate local reservations is a simple useful proxy. It avoids repeatedly choosing a replica using the same stale poll. The executor independently rejects requests when at capacity.

Follow-up: Task sizes can differ. This is not optimal cost-aware scheduling, and stale observations still require retries.

16 Why one executor if the architecture is scalable

One executor with capacity eight was the smallest measured passing configuration. Two and four replicas worsened HTTP throughput in the earlier shared-host test, despite improving some processing behavior.

Follow-up: Scalability is an ability to add capacity at the limiting stage when resources permit. It is not a rule that every replica increase must help every metric.

17 Why not directly add multiple Data replicas

The current Data process owns a local metadata-version cache and its invalidation fences. Independent replicas could serve different cached versions after one writes. Shared invalidation or removing that optimization is necessary before safe replication.

Follow-up: PostgreSQL can accept multiple clients; that alone does not make the application's caches coherent.

18 What are the autoscaling thresholds

The policy considers more than 1,024 queued events or at least 75% occupied capacity sustained demand. It can add a replica after five seconds and remove one after sixty seconds fully idle, within configurable bounds.

Follow-up: The host-side autoscaler is not automatically active in the normal Compose stack. Its load signal is not proof that the executor is the bottleneck.

Consistency and failure semantics

19 What is the most important transaction

The Data commit writes experiment versions/state, measurement rows, outbox jobs, rejected events, and the partition checkpoint together. Either they all commit or a conflict/error rolls them back.

Follow-up: The database transaction does not include the external notification HTTP request. That is precisely why the outbox exists.

20 Why store an application checkpoint when Kafka has offsets

Kafka's offset commit and PostgreSQL's transaction are separate operations. If Data commits and the Kafka acknowledgement is lost, replay must recognize that the effects already exist. The database checkpoint provides that recognition.

Follow-up: Data stores the last processed offset. Kafka is committed to the next offset to read. After processing offset 45, those values are 45 and 46.

21 What if we commit the Kafka offset first

A crash before the database transaction could leave Kafka progress ahead of the stored effects. Recovery might skip events whose history and notification job were never saved.

Show: dispatch_partition only commits after the executor's successful durable response.

22 What prevents two workers from overwriting the same state

Each proposed state update carries the version read from the snapshot. The update succeeds only if that version still matches. Checkpoint ownership is also serialized with a database row lock. A conflict rolls back and triggers reload/recomputation.

Follow-up: This is optimistic concurrency around application writes; it is not a claim that every database transaction uses SERIALIZABLE isolation.

23 How did we batch writes without losing version checks

We sorted IDs, grouped adjacent insert/update operations, and used fetchmany. Each successful operation returns a row. If returned-row count is below operation count, a version or insertion conflict occurred and the transaction aborts.

Follow-up: Grouping all inserts separately without considering lock order could undermine the original ordering discipline. The actual code groups adjacent operations after sorting.

24 What happens if a sensor message is duplicated

An identical repeated reading for an incomplete group does not add another sensor. A conflicting duplicate is invalid. A completed experiment/measurement identity is recognized on replay, including replay at new Kafka offsets.

Follow-up: For a fully completed identity, the implementation skips repeated data rather than auditing every replayed value against the original raw readings.

25 What if a sensor never arrives

The incomplete group remains in durable state. A later complete group waits behind it in timestamp order. There is a configured bound on pending groups, and a phase transition with incomplete groups is rejected.

Follow-up: The system does not fabricate a missing value. Eventual progress requires complete/correct input or an explicit future missing-sensor policy.

26 Does quarantine solve invalid input

It prevents a bad event from indefinitely blocking the Kafka partition and preserves a reason for investigation. It does not restore a missing reading, repair an invalid configuration, or make the experiment complete.

Show: Rejected-event storage/metrics and the poison-event progress test.

27 Does a Redis cache make reads eventually consistent

A cache does not inherently determine that. Here the key contains a state version, and a single Data writer invalidates metadata before and after writes while fencing concurrent fills. Reads overlapping a transaction can observe a valid earlier snapshot; reads after acknowledged writes should select the new version.

Follow-up: That correctness argument depends on the single-process invalidation ownership.

28 Why is external notification delivery at least once

The receiver may accept a request just before the sender fails to record success. When the lease expires, retry can deliver it again. The job identity is deduplicated in our database, but the receiver must honor idempotency to prevent repeated acceptance.

Avoid: “Kafka gives us exactly-once notifications.”

29 What do notification leases accomplish

They limit simultaneous ownership of due jobs and permit recovery when a worker disappears. Claims skip rows locked by another claim. A result must carry the current lease token, so an old worker cannot acknowledge a job after ownership changes.

Follow-up: A lease does not force an already-sent HTTP request to stop. A very delayed sender can still overlap an external retry.

30 Are retries always good for the ten-second target

Retries improve eventual delivery but consume time. The code retries transport errors and selected statuses with bounded delay, and records permanent request failures. A fifteen-second lease after a sender crash can already exceed the deadline.

Follow-up: Durable recovery and bounded steady-state latency are different guarantees. The current system does not promise the deadline through arbitrary failures.

31 What happens if PostgreSQL is unavailable

Data cannot durably accept the task. The executor/scheduler retry, and Kafka offsets are not acknowledged for uncommitted work. The queue can fill and intake can pause. Correctness is favored over pretending the write succeeded, but availability and latency degrade.

32 What does a consistent snapshot mean in our code

The load endpoint reads checkpoint, versioned states, and completed IDs in one repeatable-read transaction. A history cache miss obtains metadata and points in one SQL statement. Those reads avoid combining pieces from different committed moments.

Follow-up: A snapshot can be consistent without being the latest possible state at every instant during concurrent writes.

Performance and experimental reasoning

33 What is the most convincing result

The final local run of the original instructor tools: 100 concurrent experiments, ten sensors each, 1 Hz, sixty carry-out samples, and 100 offered HTTP/s. All 610 required notifications arrived, maximum 5.641 seconds; all 202 complete history comparisons were correct; achieved HTTP rate was 97.310/s.

Follow-up: One warmup ended before the main concurrency. This is historical local evidence, not a blanket hosted-grading pass.

34 What failed before that result

The initial instructor stress run received all 632 notifications and had correct histories, but three notifications exceeded ten seconds, maximum 10.716 seconds. Achieved HTTP throughput was 86.848/s. Per-experiment state-write round trips were the concrete optimization target.

35 Why did the expected notification count change

The simulator produces random temperature trajectories, and notifications depend on transitions. Same configured workload does not imply the same number of excursions. The final run required 610, and all 610 were received.

Follow-up: This is why we avoid presenting a single before/after pair as a precisely isolated causal experiment.

36 Is the performance gain statistically proven

No repeated randomized A/B series is preserved. The observed maximum fell 47.36% and HTTP rate rose 12.05%, but randomness, resource state, and workload history can contribute. The mechanism is plausible, the full tests passed, and repeat runs would strengthen attribution.

37 What does keeping up with HTTP mean

Valid completions must track the offered workload over the same time window without steadily growing queues or hidden drain time. Error rate and latency also matter. Our local 95% rule is explicit engineering acceptance, not a percentage supplied by the professor.

Follow-up: A generator's "effective rate" is not automatically the server's validated completion rate.

38 Why use independently paced requests

If a client waits for every response before scheduling another request, overload can reduce offered load and conceal a growing service-time problem. Our deterministic harness schedules independently and counts client-queue time and drain time.

Follow-up: The load generator itself can saturate on the same host, so its offered/achieved rates must be observed too.

39 Is p99 below ten seconds sufficient

No. At least one notification can exceed the deadline while p99 remains below it, and a missing notification may have no latency sample at all. We need required/received counts and a strict maximum below ten seconds.

Follow-up: The current harness accepts the exactly-ten-second boundary; the rubric says below ten. The historical passing maximum is well below both.

40 Why did more executors hurt

The added execution concurrency competed for the same CPU and downstream Data/database capacity. Earlier matrix results showed achieved HTTP around 93.4–93.6/s with two/four executors versus 98.077/s with one.

Follow-up: We observed the effect; separating CPU from database contention precisely needs further timing/profiling. Do not pretend the matrix alone measured every cause.

41 How would you pinpoint a bottleneck

Compare demand and output at each stage. Queue growth with high CPU suggests compute contention; queue/occupancy with database waits suggests Data writes; slow HTTP with healthy ingestion suggests history reads; current history with an aging outbox suggests delivery. Confirm with a targeted experiment.

Follow-up: More replicas are a hypothesis-driven intervention, not the first answer to every latency problem.

42 What is the theoretical concurrency limit

With sixteen partitions and executor capacity eight, at most min(16, 8×replicas) partition tasks can be active under this scheduler design. Throughput also depends on service time, batch size, skew, and shared resources.

Follow-up: Four executors advertising 32 slots do not create 32 independent partitions.

43 How do sensor count and experiment duration change the workload

At E experiments, S sensors, and T seconds/sample, input rate is E×S/T and aggregate rate is approximately E/T. Duration grows retained history and query size. At 100 experiments and 1 Hz, continuous carry-out yields 8.64 million aggregate points per day.

Follow-up: “Only 100 experiments” does not imply bounded storage or arbitrary sensor capacity.

44 What should we test next

Restore and reproduce the baseline, retest old failing fixed profiles on the final code, then measure continuous changing HTTP/event rates, burst recovery, skew, cache warmth, and longer history/soak behavior. Record all runs, including failures, and change one factor at a time when diagnosing causes.

Operations, costs, and project ownership

45 What did the Docker cleanup remove

It killed and removed containers and removed image references; those application processes and their writable container layers are gone. Named volumes remain, but that alone does not verify database contents or recovery. No containers were present in the November audit.

Follow-up: Docker describing an unused volume as reclaimable does not mean it is disposable. Restore and validate before saying the application is healthy.

46 What do we listen to during the professor's demo

The shared experiment topic, using the demo overlay and group15-temperature-demo-v1. Notifications go to the external course endpoint with the supplied token. Local instructor tests use course-test-events and a local notification validator.

Follow-up: Source configuration shows intended mode. Actual partition assignment and fresh processing show that it is running in that mode.

47 What was wrong with external Kafka TLS

The public hostname did not match the broker certificate names. The demo-specific setting disabled hostname matching while retaining CA validation and client-certificate authentication. Strict matching remains the general default.

Follow-up: Encryption and CA trust do not replace hostname verification. The preferred external fix is a certificate valid for the public name, after which strict mode should be restored.

48 Which parts of the dashboard are ours

The port-3004 operational dashboard is ours. The port-3009 grading view derives from the original instructor dashboard with local bindings. The bottom Latest local test panel, ID 1000, was added by us and reflects explicit runner checks. The hosted Grafana belongs to the course.

Follow-up: No-data green panels are not evidence. A pinned historical time window is useful, but it must not be confused with current live health.

49 How do we calculate costs and what is missing

Configured VM cost/hour plus measured database GB times configured storage GB-month price divided by 730. Rates default to zero with an explicit unconfigured indicator. It omits allocated unused storage, network, monitoring, and external infrastructure unless separately accounted for.

Follow-up: The full-disk incident shows why actual filesystem usage and recoverability matter even when the database-only cost estimate is small.

50 What did you personally contribute

Give a truthful concrete example with an artifact: code, test, operational recovery, benchmark analysis, documentation, or a design investigation. Explain the decision and how you checked it. Do not invent a division of labor based on degree labels or claim ownership of work you cannot explain.

Follow-up: Every member should still explain the rest of the architecture because the rubric assesses individual knowledge across the project.

Worked exercise 1 Sensor means and excursions

Use three sensors and range 25–26 inclusive. All rows represent complete groups with matching timestamp/hash.

Phase and sample	Sensor temperatures	Mean	Notification	Appears in required history
Stabilizing S1	20, 21, 22	21	None	No
Stabilizing S2	24, 25, 26	25	Stabilized	No
Running M1	24, 26, 28	26	None	Yes
Running M2	26, 27, 28	27	OutOfRange	Yes
Running M3	27, 28, 29	28	None	Yes
Running M4	24, 25, 26	25	None	Yes
Running M5	22, 23, 24	23	OutOfRange	Yes

There are five carry-out history points, three outside-history points (M2, M3, M5), two carry-out notifications, and one stabilization notification. The first excursion closes at M4. If M5 remains outside until termination, its interval closes at termination.

Now remove sensor C from M2. No M2 mean or notification can be emitted. M3, even if complete, waits behind the earlier partial group. This is a useful example of correctness requiring input completeness.

Worked exercise 2 Transaction and replay

Start with database checkpoint 40. A task processes offsets 41 through 45 and creates an average and outbox job. Data commits checkpoint 45, but the executor's response is lost before the scheduler acknowledges Kafka.

Kafka may resend 41–45. The reloaded checkpoint shows they are already durable, so no second average or job is needed. Kafka can then commit next-read offset 46. If the original database transaction had failed before commit, the checkpoint would still be 40 and the events would be recomputed.

Now change the failure: the notification receiver accepted the job, but the sender lost its acknowledgement. Database checkpointing does not prove whether that external acceptance happened. Retry may duplicate it unless the receiver honors the idempotency identity.

Worked exercise 3 Bottleneck diagnosis

Observe three windows:

Window	Observation	First hypothesis and next check
A	CPU near saturation, scheduler queue rises, HTTP p95 rises after extra replicas	Shared-host contention; compare process CPU and reduce executor competition at the same offered load
B	Histories current, outbox oldest age rises, many delivery 503s	Receiver or network trouble; inspect delivery outcomes/timeouts rather than adding executors
C	Free executor slots, one partition far behind the others	Key skew or sequential partition work; inspect per-partition progress before adding workers

A strong answer says what additional observation would falsify the hypothesis. It does not diagnose only from one correlated graph.

Source navigation drill

Before the exam, each member should locate and explain these without searching the whole repository blindly:

File	Explain aloud
engine.py	Complete grouping and previous_out_of_range
executor.py	prepare_commit, replay shortcut, conflict loop
scheduler.py	dispatch_partition, offset commit, pause/resume
data.py	Atomic commit, fetchmany/version check, metadata fence, claim/result
notification.py	Lease acknowledgement and external duplicate window
schema.sql	Uniqueness and persistence boundaries
compose.demo.yml	Topic, consumer group, credentials, notification destination
loadtests/course.py	Real-load pass conditions and the added panel
tests/test_data_integration.py	Stale-middle-version rollback
benchmarks/2026-10-09/course-stress.json	Workload and measured result

The architecture report explains the mechanisms; the performance report links each numerical claim to its source.

Team rehearsal and presentation plan

Group 15 • Final assessment preparation • 3 November 2026

The group has complementary backgrounds in Computing Science, Data Science, and Business Informatics. Use those strengths to improve explanations, but train every member to answer across the whole project. The rubric explicitly evaluates individual understanding and permits redirected questions.

The speaking roles below are proposed rehearsal responsibilities, not a historical division of implementation work. Record actual contributions separately and truthfully.

A common standard for every member

Before the exam, each person should be able to do all six tasks without notes:

1. Draw the five application responsibilities and identify PostgreSQL as the authority.
2. Follow one measurement from Avro decoding through complete-sensor averaging to durable history and notification.
3. Explain the database/Kafka acknowledgement gap and the distinct external-notification duplicate window.
4. State the successful workload and results with correct units, dates, and scope.
5. Diagnose at least two different bottlenecks and select different responses for them.
6. Explain the current recovery status, a limitation, and a proposed improvement without claiming it is already implemented.

A member who can only explain “their” component is not ready under this rubric.

Use the three backgrounds productively

Preparation lead	Lead topic	Cross-training obligation
Computing Science	Ordering, optimistic concurrency, atomic transactions, failure cases, safe scaling	Explain experimental limitations and costs in accessible language
Data Science	Workload design, offered versus achieved rate, percentiles/maxima, before/after inference, repeatability	Walk through checkpoint and outbox code
Business Informatics	Requirement traceability, experiment value, operational risk, cost scope, incident/recovery decisions	Calculate a sensor example and explain cache/version correctness

Each lead should teach the other two, who then explain the topic back using a new example. Correct the mechanism, not just the terminology.

Four preparation sessions

Session 1 Shared mental model

Spend 45–60 minutes on the architecture and worked sensor example. Each person draws the system independently; compare ownership, public interfaces, and data flow. Introduce a duplicate sensor, a missing sensor, and a phase change with incomplete input.

Finish by asking a different member to explain why the outside-history count differs from the notification count. Success means all three produce the correct outputs, not merely agree with the lead.

Session 2 Failure and code walkthrough

Spend 60 minutes opening the actual files. Rotate explanation of engine.apply, executor.prepare_commit, scheduler.dispatch_partition, Data.commit, Data.claim/result, and notifier.deliver.

Inject failures verbally before commit, after commit but before Kafka acknowledgement, and after external acceptance. Then explain the stale-middle-version regression and why it protects the performance optimization.

Success means each member can identify both the relevant function and the invariant it protects.

Session 3 Evidence and bottlenecks

Spend 45–60 minutes reading the raw JSON results. Ask one member to state the workload, another to calculate a derived quantity, and the third to challenge the claim.

Use these prompts: Why did 632 become 610? Which run has HTTP p95 0.689 seconds? What does 202 count? Why is 97.310/s different from a target-rate metric of 100? Why do old failing profiles need a rerun after optimization?

Finish with the three bottleneck windows in the oral handbook. Success means distinguishing measured observations from plausible explanations and future tests.

Session 4 Mock assessment

Use a ten-minute practice presentation followed by twenty minutes of randomly redirected questions. Ten minutes is a rehearsal format, not a confirmed exam duration. Shorten or expand the modules to the professor's allotted time.

One member acts as examiner, one answers, and one checks correctness/evidence. Rotate after each question. Repeat once so no member remains the examiner for all technical questions.

A ten-minute presentation outline

Time	Speaker for this rehearsal	Message and visual
0:00–1:00	Business Informatics	Problem, monitoring-only scope, history and timely notification, rubric targets
1:00–3:00	Computing Science	Architecture diagram, responsibility boundaries, one complete measurement
3:00–4:30	Computing Science	Atomic checkpoint/state/history/outbox; one crash example
4:30–6:30	Data Science	Exact workload, failed-to-passing comparison, what the improvement changed
6:30–8:00	Data Science	Replica matrix and bottleneck diagnosis; why more workers did not always help
8:00–9:15	Business Informatics	Storage incident, recovery, cost scope, operational limits
9:15–10:00	Any member	Demonstrated result, remaining variable-load evidence, next targeted step

Prepare eight simple slides or views corresponding to these topics. Use the generated architecture and performance figures. Show a raw result or code function when it helps answer a specific question; do not scroll the complete code appendix during the main explanation.

Suggested opening

“We monitor experiments by averaging all allocated sensors for each sampling instant, retaining carry-out history, and notifying required temperature transitions. We separated the ingestion, computation, storage, query, and delivery responsibilities. Our design uses a database transaction to keep state, history, notification obligations, and Kafka progress consistent. We then measured the design, found a database overhead problem, and improved it while preserving the consistency checks.”

Suggested performance explanation

“The first full instructor-tool test had correct history but three notifications above ten seconds and insufficient achieved HTTP rate. We reduced sequential database round trips by pipelining state writes, with all version checks inside one transaction. The next run, under the same settings, received all 610 required notifications with a 5.641-second maximum and 97.310 successful HTTP requests per second. The simulator is random, so this is an observed improvement rather than a controlled repeated causal estimate.”

Suggested limitation explanation

“Scalability depends on the limiting resource. Our worker abstraction can scale, but extra workers on a saturated shared VM competed with reads, and the Data process's local metadata cache currently prevents casual replication. We would measure changing workload on the final implementation and address the observed bottleneck first. We also need operational storage management because a full disk can stop every service on this host.”

If the deployment has not yet been restored, say so directly. An old screenshot is historical evidence, not a substitute for a working live demo.

Random redirection drills

Ask these in a different order every round:

- Business Informatics member: why can external notification delivery be duplicated?
- Data Science member: which four categories of effect commit with the checkpoint?
- Computing Science member: why does one before/after pair not isolate a causal speedup?
- Business Informatics member: what makes multiple Data replicas unsafe now?
- Computing Science member: why is the cost metric not the provider's bill?
- Data Science member: what happens if the last sensor of an old group arrives after a newer group is complete?
- Any member: what can an empty green dashboard establish?
- Any member: what is the first action when CPU is low but outbox age rises?
- Any member: what did the Docker commands remove, and what is still unverified?
- Any member: what evidence is missing to support Excellent under variable HTTP traffic?

The observer should record the mistaken mechanism or unsupported claim, then point to the relevant function or artifact. Re-answer after five minutes using a different concrete example.

Self-assessment without inventing a grade

Use this preparation scale for each topic: 0 = cannot explain; 1 = repeats terms but cannot apply them; 2 = correct explanation plus worked example; 3 = explains mechanism, evidence, tradeoff, and counterexample.

Topic	Computing Science	Data Science	Business Informatics
Requirements and complete-sensor averaging	Unassessed	Unassessed	Unassessed
Kafka ordering and replay	Unassessed	Unassessed	Unassessed
Transactions and optimistic concurrency	Unassessed	Unassessed	Unassessed
Outbox, leases, and external duplicates	Unassessed	Unassessed	Unassessed
Cache correctness and Data scaling	Unassessed	Unassessed	Unassessed
Measurement and experimental limits	Unassessed	Unassessed	Unassessed
Bottleneck diagnosis	Unassessed	Unassessed	Unassessed
Demo recovery and costs	Unassessed	Unassessed	Unassessed

This is a rehearsal tool, not the professor's numeric grade. Aim for no topic below 2 for any member and level 3 on each member's lead topic.

Actual contribution record

Before submission, fill a factual record with member name, concrete task, artifact/commit/result, decision made, verification, and what the member can demonstrate. Include code, experiments, analysis, operations, and documentation where they happened. Use dates when supported by the record.

If a contribution cannot be substantiated, describe the actual learning or review work rather than manufacturing authorship. A good oral answer shows understanding of how an artifact was produced and validated.

Responding when uncertain

Take a moment, identify the assumption, and reason from the implementation. For example: “The worker is stateless with respect to experiment ownership; the partial group is in Data. I would check the Data commit result and checkpoint before deciding whether replay is required.”

If the source or evidence does not settle a question, say what is known and what you would inspect. “The matrix shows a throughput loss with more replicas; it does not isolate database wait from CPU competition” is stronger than inventing a profiler result.

The objective is not memorizing all fifty answers. It is being able to derive the right answer from requirements, state ownership, failure boundaries, and evidence.

Recovery and demo runbook

Group 15 • Final assessment preparation • 3 November 2026

The 3 November audit found zero containers after the disk cleanup. Named PostgreSQL, Kafka, and monitoring volumes still exist, but service recovery and data integrity remain unverified. Source files match the October snapshot and 42 unit tests pass. The core stack must be restored before a live demo can satisfy the rubric.

The commands below are recovery instructions for the team. No containers were recreated, images rebuilt, volumes deleted, or load generators started while preparing these reports.

What the cleanup changed

Docker kill stops running containers abruptly. Docker rm removes containers and their writable layers. Docker rmi removes images or image references; it does not remove source files or ordinary named volumes. An abruptly stopped PostgreSQL instance may need its normal crash recovery when reopened.

The commands listed by the user did not request volume deletion. The audit confirmed the named volumes exist; it did not open the database and prove that all rows survived. Some untagged runtime/build artifacts and build cache remain, but the normal application image tags are absent.

Retained volume	Docker-reported size at inspection	Role
cec-temperature_postgres-data	1.653 GB	Application experiment state, history, checkpoints, outbox
cec-temperature_course-db-data	49.08 MB	Local instructor ground truth and accepted notifications
cec-temperature_kafka-data	2.14 GB	Local Kafka state and retained events/documents
cec-temperature_prometheus-data	87.92 MB	Operational metrics
cec-temperature_course-prometheus-data	655.2 MB	Local grading metrics
cec-temperature_grafana-data	52.43 MB	Operational Grafana state
cec-temperature_course-grafana-data	52.59 MB	Local grading Grafana state

Docker reported about 1.422 GB of build cache and a large untagged build image. “Reclaimable” means unused by a current container, not unimportant. Since all containers were removed, even the application database volume appears unused.

Step 1 Preserve data and inspect capacity

From the repository root:

```
df -h .
sudo -n docker system df
sudo -n docker ps -a
sudo -n docker image ls -a
sudo -n docker volume ls
```

Preserve the named data volumes before more cleanup. A cloud-disk snapshot or a backup to a separate mounted disk/off-host destination avoids filling the already constrained root disk. A stopped PostgreSQL volume can be preserved as a physical copy, but a normal logical dump is preferable once the server has recovered. A same-disk copy is not protection against losing that disk.

Do not use volume pruning or Compose down -v for this recovery. Do not delete directories inside a PostgreSQL volume to free space. Retained Kafka and Prometheus data may be less essential than experiment history, but any deletion still needs an explicit retention decision and preservation of required evidence.

Build cache can be regenerated, whereas the application database cannot be reconstructed reliably from an incomplete Kafka retention window. Pruning build cache may force another long Rust compile. Inspect first and choose the smallest safe cleanup or add capacity before rebuilding. Reports and source are small compared with these Docker artifacts.

The local broker is configured for 24-hour retention, operational Prometheus for fifteen days, and course Prometheus for seven days. October metrics/events may be removed under retention when services resume. The saved benchmark JSON files remain the historical evidence even if an old dashboard time range no longer has live datasource samples.

Step 2 Choose the recovery mode

Objective	Compose selection	Kafka and notifications
Professor demo	compose.yml + compose.demo.yml	External experiment topic and external notification endpoint
Deterministic local development	compose.yml + compose.local.yml	Local experiment topic and notification double
Original instructor self-test	Base + local + course-test overlay, managed by loadtests.course	Local course-test-events and original Rust notification validator

These modes share the cec-temperature application project and database. Choose one intentionally. Starting local testing changes the application's topic and destination; it does not run an independent second application automatically.

If the exam is close, restore the core demo services first. They do not require compiling the Rust test tools or running local Kafka. Plan a separate local verification window before returning to demo mode.

Step 3 Rebuild and inspect the core state

Check that the existing credentials and database-password configuration are available. Reusing a PostgreSQL volume requires the same database credentials; setting a new POSTGRES_PASSWORD variable does not reset an existing database role.

The source already has a virtual environment. If it is still usable, the lightweight check is:

```
.venv/bin/python -m pytest -q -m 'not integration'
```

Prepare the narrow password file without printing its contents, then build the application/gateway:

```
.venv/bin/python scripts/prepare-monitoring-credentials.py group15-credentials
sudo -n docker compose -f compose.yml -f compose.demo.yml config --quiet
sudo -n docker compose -f compose.yml -f compose.demo.yml build data api
```

Start the database, cache, Data, executor, and API first, leaving event consumption and outbound delivery for the next step:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml up -d \
  postgres redis data executor api
sudo -n docker compose -f compose.yml -f compose.demo.yml ps
curl --fail http://127.0.0.1:3003/health
sudo -n docker compose -f compose.yml -f compose.demo.yml logs --tail 100 postgres data
```

Wait for health and review database recovery before relying on the data. Then inspect row counts and outbox status without dumping secret-bearing payloads:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
```

```
  "SELECT count(*) AS experiments FROM experiments;"
sudo -n docker compose -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
  "SELECT status, count(*) FROM notification_outbox GROUP BY status;"
```

Compare known experiment IDs/history against preserved evidence where possible. Take a logical database backup to the chosen external destination after successful recovery.

Inspect pending jobs before changing their notification destination. Because local and demo modes share the application database, an unfinished local test job is not automatically appropriate for the real endpoint. Preserve and diagnose unexpected jobs rather than blindly deleting history or starting delivery to a different validator.

Step 4 Verify external connectivity and start demo processing

The preflight is read-only: it neither publishes nor commits the live consumer's offsets.

```
.venv/bin/python scripts/check-course-kafka.py --topic experiment
```

A successful preflight establishes metadata access and a temporary consumer's assignment. It does not prove that the actual scheduler is running. After state and destination checks, start the actual processes:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml up -d scheduler notification
curl --fail http://127.0.0.1:8001/health
curl --fail http://127.0.0.1:8001/status
sudo -n docker compose -f compose.yml -f compose.demo.yml logs --tail 100 scheduler notification
```

Confirm the intended topic is experiment, the consumer group remains group15-temperature-demo-v1, and notifications go to the course endpoint. Do not change the group merely to clear a queue; a new group starts from earliest retained events and changes the checkpoint identity.

The historical course topic had sixteen partitions. Verify the current assignment rather than treating that old number as permanent. A growing queue immediately after recovery may represent catch-up; assess its trend, source timestamps, and notification age before interpreting fresh latency.

If TLS fails, distinguish port access, CA trust, hostname matching, keystore readability, authentication, and topic authorization. The demo overlay contains the known hostname exception; restore strict matching when valid public-name certificates are available. Do not print the token or keystore password during troubleshooting.

Step 5 Restore observation and check public access

When resources permit, start operational monitoring:

```
sudo -n docker compose -f compose.yml -f compose.demo.yml \
  -f compose.monitoring.yml up -d prometheus node-exporter grafana
```

The public API is port 3003. Operational Prometheus is port 3008, used for the course's optional infrastructure metrics. Operational Grafana is loopback 3004. Access it through SSH forwarding from the laptop:

```
ssh -N -L 3004:127.0.0.1:3004 ubuntu@YOUR_VM_PUBLIC_IP
```

Use the existing SSH key/options appropriate to the VM. Verify API reachability from outside the VM as well as localhost; local health does not establish routing/firewall reachability for the professor.

For a known experiment, query both required endpoints:

```
curl --fail --get http://YOUR_VM_PUBLIC_IP:3003/temperature \
  --data-urlencode 'experiment-id=KNOWN_EXPERIMENT_ID' \
  --data-urlencode 'start-time=0' \
  --data-urlencode 'end-time=2000000000'
curl --fail --get http://YOUR_VM_PUBLIC_IP:3003/temperature/out-of-range \
  --data-urlencode 'experiment-id=KNOWN_EXPERIMENT_ID'
```

Use the course-hosted Grafana and a current workload/time range to observe the professor's results. Do not call an old pinned local test window current evidence.

Step 6 Revalidate locally in a separate test window

The source/configuration syntax checks succeeded during this report preparation, but running the full checks still requires recovered containers and enough space.

Use this window outside the professor's workload. Before switching modes, stop intake, let outstanding jobs drain to their current validator, and confirm that no pending jobs would be sent to the wrong destination.

For the deterministic local stack:

```
sudo -n docker compose -f compose.yml -f compose.local.yml up -d
TO_TEST_STACK=1 .venv/bin/python -m pytest -q
.venv/bin/python -m loadtests.stress --experiments 3 --sensors 3 \
  --duration 3 --http-rps 20 --output test-results/recovery-smoke.json
```

Integration tests create isolated test databases; they are not a substitute for verifying the retained production database. Do not benchmark concurrently with a build, test suite, or recovery exercise.

For the actual instructor tools:

```
.venv/bin/python -m loadtests.course run
.venv/bin/python -m loadtests.course run --experiments 100 --sensors 10 \
  --samples 60 --http-rps 100 --http-seconds 60
.venv/bin/python -m loadtests.course check
.venv/bin/python -m loadtests.course stop
```

The first missing Rust image may require a long compile and substantial disk space. Remaining intermediate images may help the cache, but should not be blindly retagged as completed runtimes; an intermediate can lack the final setup step.

The runner's prepare stage refreshes the generated local thirty-day JWT. It reuses suitable local certificate/key files and generates the workload/datasource configuration. The supplied external course token has a separate lifecycle.

Forward port 3009 to see the local grading dashboard. The added bottom Latest local test panel gives explicit runner status. Inspect actual counts and maximum latency as well as that status; apply the rubric's strict less-than-ten-second interpretation.

After local tests, stop the generators and explicitly restore the demo overlay. Confirm topic, destination, actual assignment, and public access again. Stopping the generators alone does not switch application configuration back to the course.

Prevent another full-disk incident

Set operational thresholds for free space and growth, with enough reserve for rebuilding images, database writes, and a test run. A proposed starting policy is warning below 25% free and an operational stop on optional builds/load below 15% free, then tune against measured growth. These are planning thresholds, not implemented alerts or a guarantee.

The existing alert rules cover latency, rejected events/jobs, and scrape health. They do not yet include filesystem-capacity alerts or external Alertmanager delivery. Node exporter exposes filesystem data that can support a new rule.

Periodically account for application history/indexes/outbox, Kafka retention, monitoring retention, logs, images, and build cache. A fixed Redis limit does not constrain these. Preserve historical experiment data through an agreed archive/export policy rather than silently deleting the user's requested trace.

Do not assume that removing containers frees their named data volumes. The November inspection demonstrated that distinction: the database and Kafka volumes alone retained several gigabytes after all containers were gone.

Final readiness check before the exam

A team member should be able to show, from a fresh record:

- A working public API and known correct history.
- The actual scheduler's current topic/group/assignment and a stable or draining queue.
- Correct external notification destination and an inspected outbox.
- Fresh nonzero workload validation, with required/received counts and strict latency.
- Variable-load evidence and honest capacity limits.
- Current disk headroom and a way to preserve/recover data.
- A current dashboard window, with local and hosted results clearly identified.
- All three members ready to explain both the design and one failure case.

The readiness audit is the starting point for recovery, not a completion record.

Exam quick reference

Group 15 • Final assessment preparation • 3 November 2026

Rubric: final demo 70%, architecture 30%. Excellent requires a working, consistent service under variable event load, notifications **below 10 seconds**, keeping up with variable HTTP load, and identifying bottlenecks under load. Questions can go to any member.

Current audit, 3 November: zero containers after cleanup; named volumes exist but recovery is unverified. Source unchanged from the October report. Fresh unit tests: 42 passed, ten integration tests deselected.

The architecture in one sentence

Kafka -> ordered partition scheduler -> stateless executor -> Data transaction -> PostgreSQL; Redis caches history, Caddy exposes the API, and a separate notifier drains durable outbox jobs.

The atomic commit joins state, history, jobs, rejected input, and checkpoint. Kafka is acknowledged afterward. Losing the acknowledgement causes safe replay. External HTTP acceptance has its own crash window, so delivery is at least once unless the receiver honors idempotency.

Facts to remember

Fact	Correct value or meaning
Demo input	experiment topic; group15-temperature-demo-v1
Public API	Port 3003; /temperature and /temperature/out-of-range
Local panels	Operational 3004; instructor-style 3009
Default execution	One executor, capacity eight; sixteen historical partitions
Task buffering	256-event default batch; pause at 8,192 queued/in-flight events
Notifications	0.1-second empty poll, concurrency 32, timeout 2 seconds, lease 15 seconds
Sensor average	All allocated sensors for the same measurement ID
Boundaries	Temperature thresholds and history time bounds are inclusive
Alert transitions	Stabilized on entry/re-entry during stabilization; one OutOfRange per carry-out excursion
History	Carry-out only; outside history includes every outside point

The evidence story

The final local instructor test used **100 experiments × 10 sensors × 1 Hz**, sixty carry-out samples and 100 offered HTTP/s. One warmup completed before the main experiments.

Measure	Before state-write pipelining	After
Required / received notifications	632 / 632	610 / 610
Above-ten-second notifications	3	0
Maximum latency	10.716 s	5.641 s

Measure	Before state-write pipelining	After
Achieved HTTP/s	86.848	97.310
Complete history comparisons	202 correct	202 correct

The random simulator explains the changed notification count. The observed improvement is 47.36% lower maximum latency and 12.05% higher HTTP throughput, not a repeated controlled causal estimate.

What changed: fetchmany pipelines adjacent sorted state writes. Every version check must return success or the entire transaction rolls back. The regression test makes the middle of three versions stale.

Earlier replica tests: one executor achieved 98.077 HTTP/s; two/four achieved about 93.4–93.6. More workers competed for shared resources. The earlier 200-HTTP/s and twenty-sensor/100-HTTP/s profiles missed the local throughput criterion and need retesting after the later optimization.

Five answers that protect against overclaiming

1. **Exactly once?** Database effects are deduplicated; external notification acceptance is not exactly once without receiver cooperation.
2. **Scale Data?** Its local metadata-version cache requires shared invalidation or removal before multiple processes are safe.
3. **Green dashboard?** Require real load, correct history, required/received alerts, maximum latency, and throughput; no-data green is insufficient.
4. **All workloads passed?** A stated historical local workload passed. Variable professor workloads and current live health need their own evidence.
5. **All data survived cleanup?** Volume names survive; database recovery and content integrity still need verification.

Diagnose before scaling

High CPU + growing processing and HTTP queues: inspect shared-host contention. Slow commits + database waits: inspect Data/database. Current history + old outbox/retries: inspect notification endpoint. Idle workers + one lagging partition: inspect key skew. Growing disk: preserve data and recover capacity, not more workers.

Input rate is E×S/T. Aggregate rate is E/T. Concurrency is bounded by min(partitions, replicas×capacity). None of these formulas by itself gives measured sustainable throughput.

Before you speak

Answer with **decision -> mechanism -> evidence -> limitation**. Distinguish what exists, what was measured, and what is proposed. Every member should explain the transaction, the performance result, a bottleneck, and the next improvement.

Source and details: rubric, design, results, questions, recovery.